Relatório – Gerenciamento de Farmácia

Oct 21, 2026 · @Victor

Visão geral

O sistema controla o estoque de medicamentos de uma farmácia em uma aplicação desktop com interface gráfica, e guarda os dados em um arquivo CSV.

- **Tecnologias:** Java 21, JavaFX (FXML), Maven e Stream API.
- **Pacote:** `com.example.gerenciamentofarmacia`.
- **Repositório:** github.com/victorhugoEngineer/GerenciamentoFarmacia

Funcionalidades implementadas

O sistema cobre o ciclo completo de cadastro, consulta e relatórios, em duas abas.

Aba Medicamentos

- **Cadastro** de medicamento com código, nome, descrição, princípio ativo, validade, quantidade, preço e indicação de controlado, junto com os dados do fornecedor (CNPJ, razão social, telefone, e-mail, cidade e UF).
- **Consulta** por código, sem diferenciar maiúsculas de minúsculas.
- **Listagem** de todos os medicamentos em uma tabela, ordenados por nome.
- **Exclusão** do item selecionado, com pedido de confirmação.
- **Persistência em CSV** separado por ponto e vírgula: o arquivo é lido ao iniciar e regravado a cada cadastro ou exclusão.
- **Validação** dos dados antes de gravar, com todos os erros mostrados juntos em uma janela:
  - código com 7 caracteres alfanuméricos e único;
  - nome com ao menos 3 caracteres;
  - validade não pode ser data passada;
  - quantidade não negativa e preço maior que zero;
  - CNPJ com verificação dos dígitos, e-mail com formato válido e UF com 2 letras.

Aba Relatórios (Stream API)

- Medicamentos próximos ao vencimento (30 dias), do mais próximo ao mais distante: `filter` e `sorted`.
- Estoque baixo (menos de 5 unidades): `filter` e `sorted`.

- Valor total do estoque por fornecedor: `groupingBy`, `mapping` e `reducing`.
- Controlados e não controlados: `partitioningBy`.
- Valor total do estoque: `map` e `reduce`.
- Os relatórios são recalculados após cada cadastro ou exclusão.

Decisões de design

O projeto separa interface, regras e dados, para que cada parte possa mudar sem afetar as outras.

- **Camadas separadas:** as entidades (`Medicamento`, `Fornecedor`), a validação (`Validador`), o acesso ao CSV (`MedicamentoRepository`), as regras do cadastro (`MedicamentoService`), os relatórios (`RelatorioService`) e a tela (`FarmaciaController` com o FXML) ficam em classes distintas. A tela não conhece o formato do arquivo.
- **Composição:** cada `Medicamento` tem um `Fornecedor`, o que evita repetir os dados do fornecedor em atributos soltos.
- **Entidades imutáveis:** sem métodos set, os objetos só entram no sistema depois de validados e não ficam em estado inválido.
- **`BigDecimal` e `LocalDate`:** o preço usa `BigDecimal` para evitar erros de arredondamento do `double`, e as datas usam `LocalDate`.
- **Validação centralizada:** o `Validador` junta todos os erros e os lança em uma única `ValidacaoException`, assim o usuário corrige tudo de uma vez.
- **Tratamento de erros:** falhas de regra e de arquivo são capturadas na tela e mostradas em janelas de diálogo, sem derrubar o programa.
- **Persistência segura:** o CSV é escrito primeiro em um arquivo temporário e depois movido, para não corromper os dados se a gravação falhar. Se isso acontecer, a operação é desfeita na memória, mantendo lista e arquivo coerentes.
- **Formato do CSV protegido:** pontos e vírgulas e quebras de linha digitados pelo usuário são substituídos antes de gravar.
- **CNPJ:** guardado só com dígitos e exibido formatado; a validação aceita com ou sem máscara.
- **Relatórios com Stream API:** cada relatório é um método pequeno que recebe a lista e devolve o resultado. Receber a data de referência no construtor permite testá-los com datas fixas.
- **Diálogos em FXML:** erros e confirmações usam um `DialogPane` carregado de `dialogo-view.fxml`, com aparência padronizada.

Limitações e melhorias futuras: não há edição de medicamento (para alterar, exclui-se e cadastra-se de novo), e o fornecedor é digitado a cada cadastro; uma tela própria de fornecedores evitaria a repetição.